

## Region Segmentation

- Binary Regions
- Region Features
- k Means clustering
- Watershed Segmentation
- Mean Shift Segmentation

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 1

1

## Segmentation

- Split images into smaller parts
  - Preferably corresponding to (parts of) objects.
- Two main approaches
  - Region based
  - Edge based
- Video segmentation
  - Breaking video into clips
  - Segmenting objects/regions consistently over time

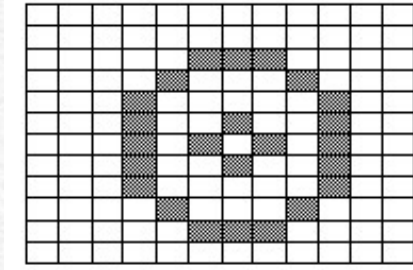
Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 2

2

## Connectivity – Paradoxes



Use pixel Adjacency to build contiguous regions

- Objects
- Background
- Holes

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 3

3

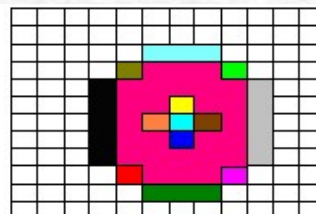
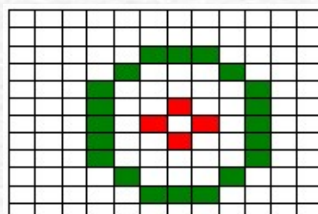
## Connectivity – 4 adjacency & 8 adjacency

Have to use either 4-adjacency or 8-adjacency

- Label each non-zero pixel...

|   |   |   |
|---|---|---|
| 3 | 2 | 1 |
| 4 | 8 | 0 |
| 5 | 6 | 7 |

|   |   |   |
|---|---|---|
| 3 | 2 | 1 |
| 4 | 8 | 0 |
| 5 | 6 | 7 |



Regions

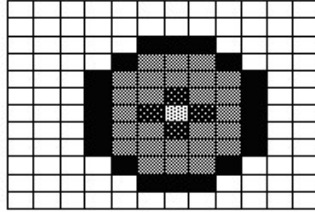
Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 4

4

## Connectivity –

What do we actually want?



One possibility

- Treat background using 4-adjacency
- Treat object using 8-adjacency
- Treat holes using 4-adjacency
- Treat objects in holes using 8-adjacency
- ...

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 5

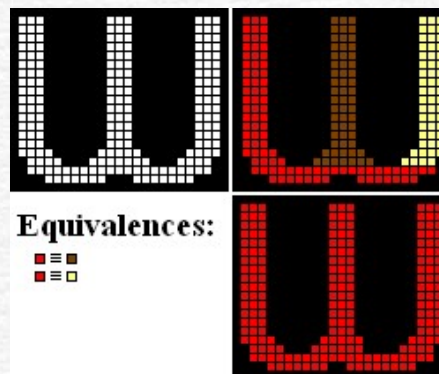
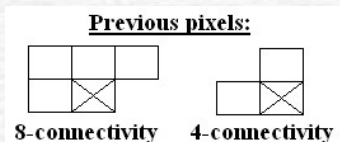
5

## Connectivity – Connected Components Analysis

Search image row by row

- Label each non-zero pixel
- If previous pixels are all background
  - Assign New Label
- Otherwise
  - Pick any label from the previous pixels
  - If any of the other previous pixels have a different label
    - Note equivalence

Relabel equivalent labels.



Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 6

6

## Connectivity – Extracting regions

|                      |                      |
|----------------------|----------------------|
| 00000000000000000000 | 00000000000000000000 |
| JJJJJJJJJJJJJJJJJJJJ | JJJJJJJJJJJJJJJJJJJJ |
| WWWWWWWWWWWWWWWWWWWW | WWWWWWWWWWWWWWWWWWWW |
| BABABABABABABABABAB  | BABABABABABABABABAB  |

```
vector<vector<Point>> contours;
vector<Vec4i> hierarchy;
findContours( binary_image, contours, hierarchy,
              CV_RETR_TREE, CV_CHAIN_APPROX_NONE );
```

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 7

7

## Connectivity – Labelling regions

|                      |                      |
|----------------------|----------------------|
| 00000000000000000000 | 00000000000000000000 |
| JJJJJJJJJJJJJJJJJJJJ | JJJJJJJJJJJJJJJJJJJJ |
| WWWWWWWWWWWWWWWWWWWW | WWWWWWWWWWWWWWWWWWWW |
| BABABABABABABABABAB  | BABABABABABABABABAB  |

```
for (int contour=0; (contour < contours.size()); contour++)
{
    Scalar colour( rand()&0xFF,rand()&0xFF,rand()&0xFF );
    drawContours( contours_image, contours, contour, colour,
                  CV_FILLED, 8, hierarchy );
}
```

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 8

8

## Region Features

- Given a segmented object / shape / region
  - We can describe it using features.
- Area
  - Number of pixels

Regions

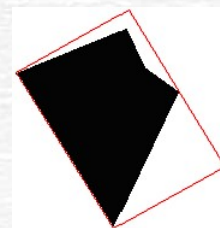
Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 9

9

## Features: Minimum Bounding Rectangle

- Turn rectangle through discrete steps
- Only one quadrant
- Metrics:
  - Length to Width ratio
    - $\text{Length} / \text{Width}$
  - Rectangularity
    - $\text{Area} / (\text{Length} * \text{Width})$
  - Convex Hull to Minimum Bounding Rectangle area ratio
    - $\text{Area inside convex hull} / (\text{Length} * \text{Width})$



```
RotatedRect min_bounding_rectangle =  
    minAreaRect(contours[contour_number]);
```

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 10

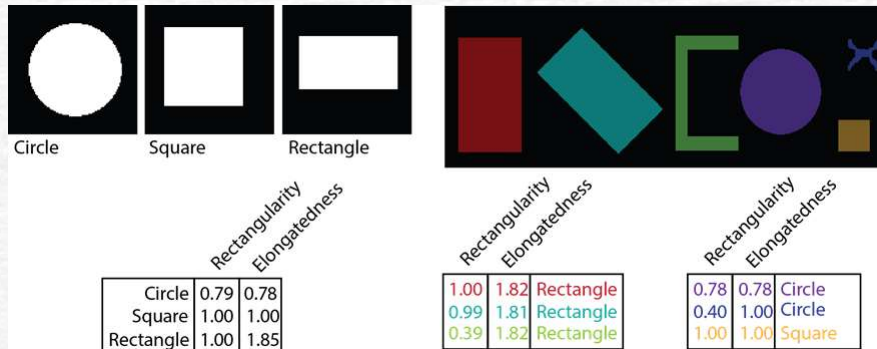
10

## Features: Elongatedness

- CANNOT BE the ratio of the length and width of the minimum bounding rectangle
- Ratio of region area divided by the square of it's thickness

• Erosion

$$elongatedness = \frac{area}{(2d)^2}$$



Regions

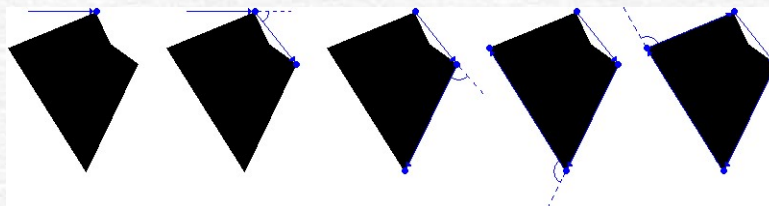
Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 11

11

## Features: Convex hull

- Smallest convex region



- Algorithm

- Start with
  - Any point on convex hull
  - Previous vector direction
- Search all other boundary points
  - Find point with least angle to previous vector
- Switch to new point and vector
- Go to 2 unless new point = start point.

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 12

12

## Features: Convex hull

```
vector<vector<Point>> hulls(contours.size());  
for (int contour=0; (contour<contours.size()); contour++)  
{  
    convexHull(contours[contour], hulls[contour]);  
}
```

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 13

13

## Features: Moments & Moment invariants

- ✓ Moments measure the distribution of shape  $M_{xy} = \sum_i \sum_j i^x j^y f(i, j)$
- ✓ Central moments:

$$\mu_{00} = M_{00} \quad \mu_{01} = 0 \quad \mu_{10} = 0$$

$$\mu_{11} = M_{11} - \frac{M_{10}M_{01}}{M_{00}} \quad \mu_{20} = M_{20} - \frac{M_{10}M_{10}}{M_{00}} \quad \mu_{02} = M_{02} - \frac{M_{01}M_{01}}{M_{00}}$$

$$\mu_{21} = M_{21} - \frac{2M_{10}M_{11}}{M_{00}} - \frac{M_{01}M_{20}}{M_{00}} + \frac{2M_{10}M_{10}M_{01}}{M_{00}M_{00}} \quad \mu_{03} = M_{03} - \frac{3M_{01}M_{02}}{M_{00}} + \frac{2M_{01}M_{01}M_{01}}{M_{00}M_{00}}$$

$$\mu_{12} = M_{21} - \frac{2M_{01}M_{11}}{M_{00}} - \frac{M_{10}M_{02}}{M_{00}} + \frac{2M_{10}M_{01}M_{01}}{M_{00}M_{00}} \quad \mu_{30} = M_{30} - \frac{3M_{10}M_{20}}{M_{00}} + \frac{2M_{10}M_{10}M_{10}}{M_{00}M_{00}}$$

- ✓ Scale invariant moments:

$$\eta_{xy} = \frac{\mu_{xy}}{\mu_{00}^{(1+x+y)/2}}$$

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 14

14

## Features: Moment invariants

- Moment invariants / Hu moments:

$$\begin{aligned}
 I_1 &= \eta_{20} + \eta_{02} \\
 I_2 &= (\eta_{20} - \eta_{02})^2 + 4(\eta_{11})^2 \\
 I_3 &= (\eta_{30} - 3\eta_{12})^2 + (3\eta_{21} - \eta_{03})^2 \\
 I_4 &= (\eta_{30} + \eta_{12})^2 + (\eta_{21} + \eta_{03})^2 \\
 I_5 &= (\eta_{30} - 3\eta_{12})(\eta_{30} + \eta_{12})((\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2) \\
 &\quad + (3\eta_{21} - \eta_{03})(\eta_{21} + \eta_{03})(3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2) \\
 I_6 &= (\eta_{20} - \eta_{02})((\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2) + 4\eta_{11}(\eta_{30} + \eta_{12})(\eta_{21} + \eta_{03}) \\
 I_7 &= (3\eta_{21} - \eta_{03})(\eta_{30} + \eta_{12})((\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2) \\
 &\quad + (\eta_{30} - 3\eta_{12})(\eta_{21} + \eta_{03})(3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2)
 \end{aligned}$$

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 15

15

## Features: Moments and Moment invariants

0 1  
2 3  
4 5  
6 7  
8 9



|   | Shape | Perimeter | Area | Bounding Box | Convex Hull    | Hu Moments | Hole Areas |
|---|-------|-----------|------|--------------|----------------|------------|------------|
| 0 | 80    | 406       | 651  | 555          | 0.17 0.00 0.00 | 175        |            |
| 1 | 101   | 274       | 522  | 429          | 0.47 0.13 0.01 |            |            |
| 2 | 111   | 347       | 570  | 528          | 0.42 0.07 0.00 |            |            |
| 3 | 121   | 351       | 522  | 530          | 0.40 0.06 0.00 |            |            |
| 4 | 85    | 347       | 592  | 442          | 0.20 0.01 0.00 | 60         |            |
| 5 | 122   | 351       | 570  | 504          | 0.43 0.06 0.00 |            |            |
| 6 | 102   | 392       | 620  | 534          | 0.23 0.01 0.00 | 76         |            |
| 7 | 94    | 267       | 551  | 394          | 0.46 0.09 0.04 |            |            |
| 8 | 65    | 438       | 651  | 571          | 0.20 0.01 0.00 | 45 62      |            |
| 9 | 101   | 389       | 589  | 524          | 0.23 0.01 0.00 | 74         |            |

```

Moments contour_moments;
double hu_moments[7];
contour_moments = moments( contours[contour] );
HuMoments( contour_moments, hu_moments );

```

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 16

16

## Features: Concavities and Holes

- Identify concavities from the convex hull
- Identify holes in the binary shape



|   | Shape | Height / Width | Hull / Box | # holes | Hole Areas | # concavities | Concavity Details |
|---|-------|----------------|------------|---------|------------|---------------|-------------------|
| 0 | 2.46  | 0.85           | 1          | 4.1     | 0          | 0             |                   |
| 1 | 4.71  | 0.58           | 0          |         | 2          | 0.4@243       | 0.03@261          |
| 2 | 2.83  | 0.85           | 0          |         | 5          | 3.6@213       | 2.5@45            |
| 3 | 2.83  | 0.82           | 0          |         | 3          | 5.7@185       | 0.2@354           |
| 4 | 2.38  | 0.62           | 1          | 0.6     | 3          | 0.9@101       | 0.1@207           |
| 5 | 2.54  | 0.85           | 0          |         | 4          | 3.1@140       | 2.4@302           |
| 6 | 2.13  | 0.79           | 1          | 1.5     | 3          | 3.1@317       | 0.03@94           |
| 7 | 2.36  | 0.57           | 0          |         | 9          | 4.3@205       | 0.1@294           |
| 8 | 2.83  | 0.80           | 2          | 1.8 1.6 | 2          | 0.3@205       | 0.2@355           |
| 9 | 2.62  | 0.81           | 1          | 1.9     | 4          | 3.0@143       | 0.1@240           |

|   | Shape | Height / Width | Hull / Box | # holes | Hole Areas | # concavities | Concavity Details |
|---|-------|----------------|------------|---------|------------|---------------|-------------------|
| 9 | 2.62  | 0.81           | 1          | 1.8     | 3          | 2.9@137       | 0.03@66           |
| 5 | 2.54  | 0.85           | 0          |         | 4          | 3.0@140       | 2.4@302           |
| 0 | 2.54  | 0.87           | 1          | 4.5     | 3          | 0.4@171       | 0.1@116           |
| 3 | 2.62  | 0.81           | 0          |         | 3          | 5.7@184       | 0.4@343           |
| 7 | 2.36  | 0.59           | 0          |         | 4          | 4.2@211       | 0.4@0             |
| 8 | 2.62  | 0.81           | 2          | 1.6 1.5 | 3          | 0.3@354       | 0.3@203           |
| 2 | 2.43  | 0.84           | 0          |         | 3          | 3.9@201       | 2.8@40            |
| 5 | 2.62  | 0.83           | 0          |         | 4          | 3.0@143       | 2.4@300           |

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 17

17

## Features: Concavities and Holes

```
vector<vector<int>> hull_indices(contours.size());
vector<vector<Vec4i>> convexity_defects(contours.size());
for (int contour=0; (contour<contours.size()); contour++)
{
    convexHull( contours[contour], hull_indices[contour] );
    convexityDefects( contours[contour], hull_indices[contour],
                      convexity_defects[contour]);
}
```

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 18

18

## Features: Perimeter Length and Circularity

- Perimeter length
  - Approximately the number of boundary elements  
`contours[contour].size()`
  - Should really take into account direction
- Circularity =  $(4 * \pi * \text{Area}) / (\text{Perimeter length})^2$

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 19

19

## Features: Summary

- Area
- Minimum Bounding Rectangle (Rectangularity)
- Elongatedness
- Convex Hull
- Moments & Moment Invariants
- Concavities & Holes
- Perimeter length & Circularity

|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |
| 4 | 5 |
| 6 | 7 |
| 8 | 9 |



|   | Shape | Perimeter | Area | Bounding Box | Convex Hull | Hu Moments     | Hole Areas |
|---|-------|-----------|------|--------------|-------------|----------------|------------|
| 0 |       | 80        | 406  | 651          | 555         | 0.17 0.00 0.00 | 175        |
| 1 |       | 101       | 274  | 522          | 429         | 0.47 0.13 0.01 |            |
| 2 |       | 111       | 347  | 570          | 528         | 0.42 0.07 0.00 |            |
| 3 |       | 121       | 351  | 522          | 530         | 0.40 0.06 0.00 |            |
| 4 |       | 85        | 347  | 592          | 442         | 0.20 0.01 0.00 | 60         |
| 5 |       | 122       | 351  | 570          | 504         | 0.43 0.06 0.00 |            |
| 6 |       | 102       | 392  | 620          | 534         | 0.23 0.01 0.00 | 76         |
| 7 |       | 94        | 267  | 551          | 394         | 0.46 0.09 0.04 |            |
| 8 |       | 65        | 438  | 651          | 571         | 0.20 0.01 0.00 | 45 62      |
| 9 |       | 101       | 389  | 589          | 524         | 0.23 0.01 0.00 | 74         |

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 20

20

## k means Clustering

### ✓ We'd like to

- identify significant colours in images
  - Concise descriptions
  - Object tracking
- reduce the number of colours in any image
  - Compression



### ✓ How do we find the best colours?

### ✓ k means clustering

- Creates k clusters of pixels
- Unsupervised learning



Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 21

21

## k means Clustering – Algorithm

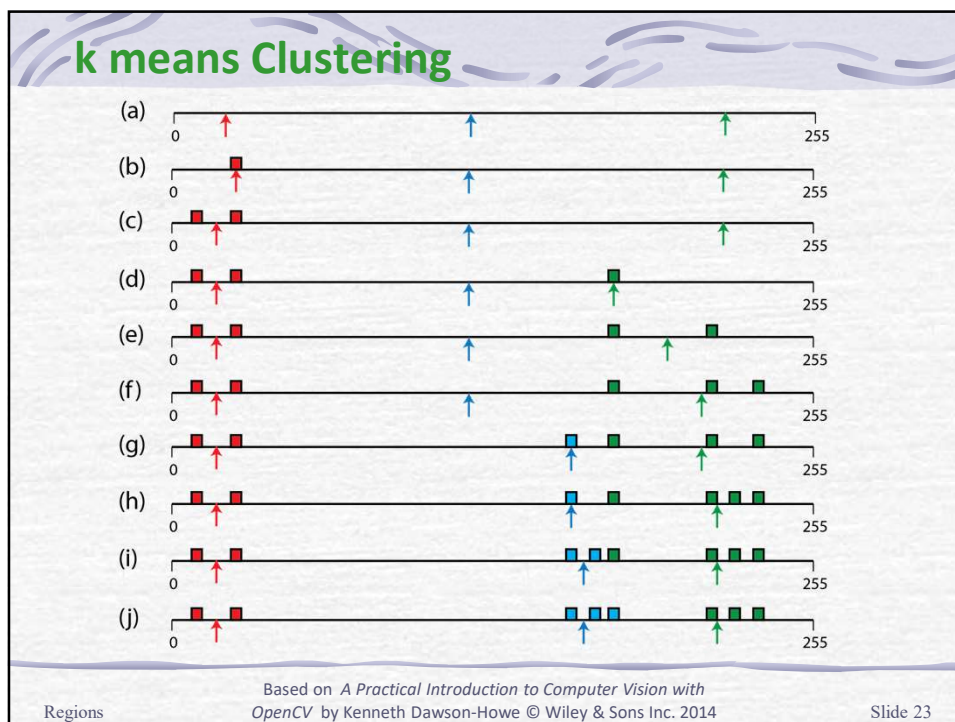
- ✓ Number of clusters (k) is known in advance (or determine the k with the maximum confidence)
- ✓ Initialise the k cluster exemplars either randomly or use the first k patterns or ...
- ✓ 1<sup>st</sup> pass: Allocate patterns to the closest existing cluster exemplar and recompute the exemplar as the centre of gravity
- ✓ 2<sup>nd</sup> pass: Using the final exemplars from the first pass allocate all patterns cluster exemplars.

Regions

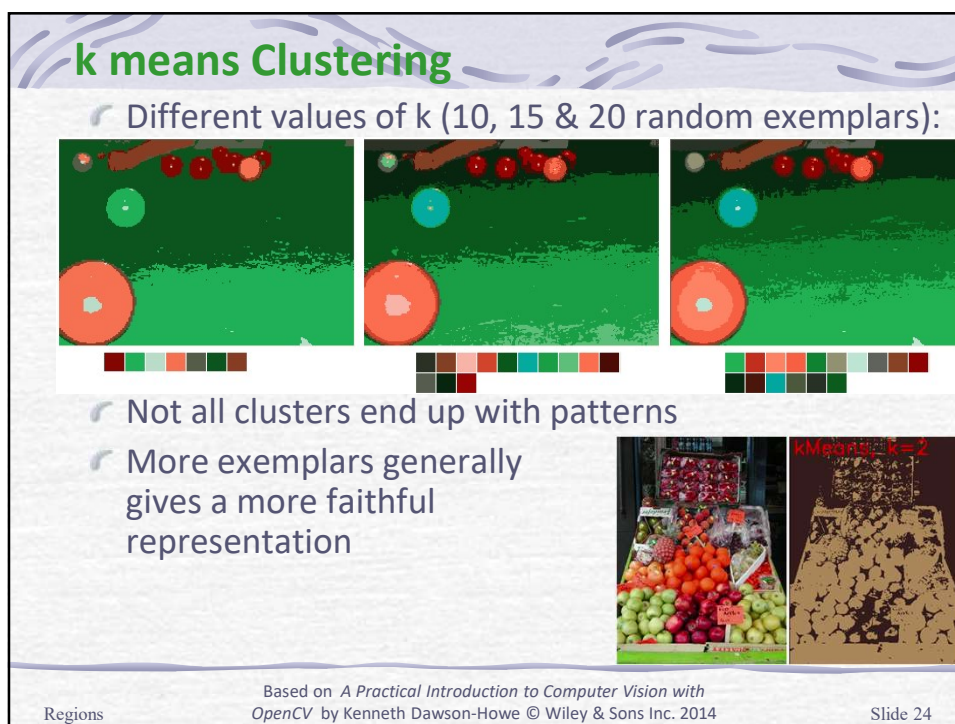
Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 22

22



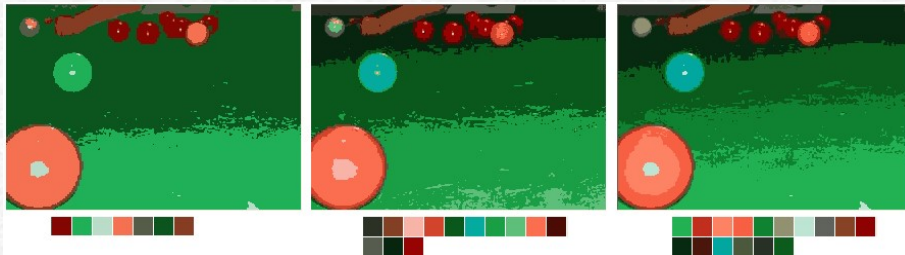
23



24

## k means Clustering

- Choosing the best number of exemplars
  - Evaluate the resulting clusters



- Davies-Bouldin index measures cluster separation:

$$DB = 1/k \sum_{i=1..k} \max_{i \neq j} ((\sigma_i + \sigma_j) / \delta_{i,j})$$

- OR Check that distributions are normal

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 25

25

## k means Clustering

- Using random exemplars gives non-deterministic results (30 random exemplars each time):



Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 26

26

## k means Clustering

```
// Store the image pixels as an array of samples
Mat samples(image.rows*image.cols, 3, CV_32F);
float* sample = samples.ptr<float>(0);
for(int row=0; row<image.rows; row++)
    for(int col=0; col<image.cols; col++)
        for (int channel=0; channel < 3; channel++)
            samples.at<float>(row*image.cols+col,channel) =
                (uchar) image.at<Vec3b>(row,col)[channel];
// Apply k-means clustering, determining the cluster
// centres and a label for each pixel.
....
```

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 27

27

## k means Clustering

```
Mat labels, centres;
kmeans(samples, k, labels, TermCriteria( CV_TERMCRIT_ITER |
    CV_TERMCRIT_EPS, 0.0001, 10000), iterations,
    KMEANS_PP_CENTERS, centres );
// Use centres and label to populate result image
Mat& result_image = Mat( image.size(), image.type() );
for(int row=0; row<image.rows; row++)
    for(int col=0; col<image.cols; col++)
        for (int channel=0; channel < 3; channel++)
            result_image.at<Vec3b>(row,col)[channel] =
                (uchar) centres.at<float>( *(labels.ptr<int>(
                    row*image.cols+col )), channel);
```

Regions

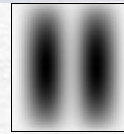
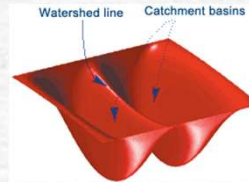
Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 28

28

## Watershed segmentation

- ☞ In geology watersheds separate different catchment basins.
  - Where the rain gets caught
- ☞ In computer vision we
  - Identify all minima.
  - Label as different regions.
  - Flood from the minima extending the regions.
  - Where regions meet we have watershed lines.
- ☞ Minimum what?
  - Greyscale
  - Gradient
  - Inverse of the chamfer distance



Images from <https://uk.mathworks.com/company/newsletters/articles/the-watershed-transform-strategies-for-image-segmentation.html> © 1994-2017 The MathWorks, Inc.

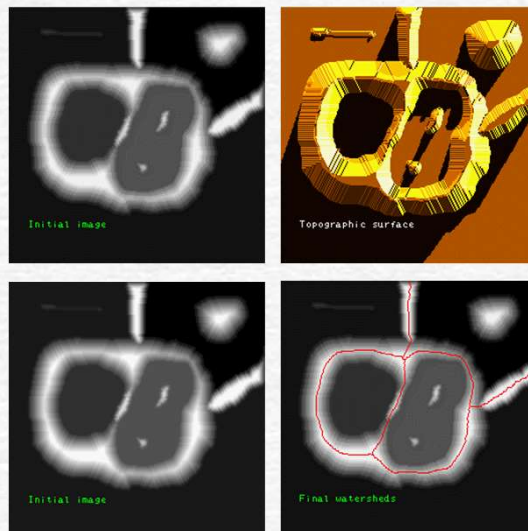
Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 29

29

## Watershed segmentation example



Images from <http://cmu.ece.cmu.edu/~beucher/watershed.html> Reproduced with permission. © 2010, Serge Beucher

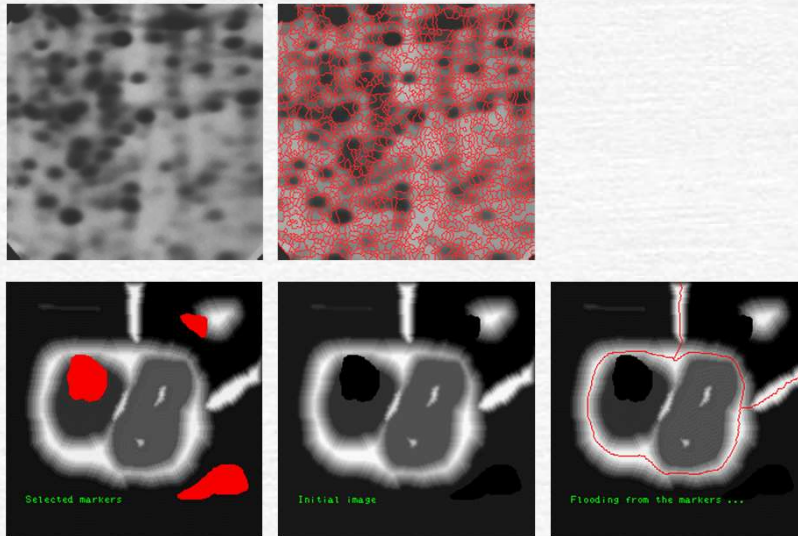
Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 30

30

## Watershed segmentation with markers



Images from <http://cvlab.epfl.ch/~beucher/wshed.html> Reproduced with permission. © 2010, Serge Beucher

Regions

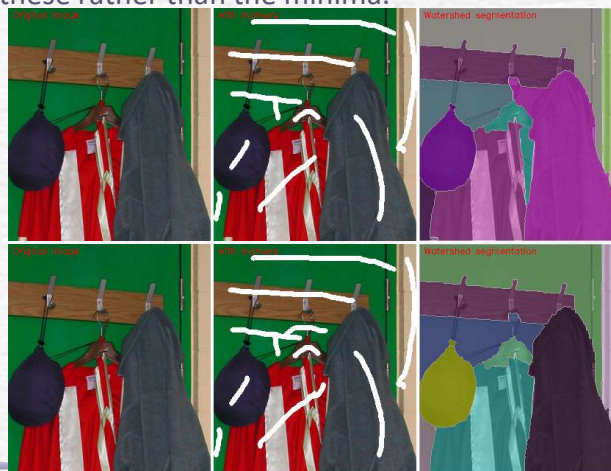
Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 31

31

## Watershed segmentation with markers

- Generally we get too many regions
- Use a priori labels to identify 'objects' and expand from these rather than the minima.



Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 32

32

## Mean Shift Segmentation

### k-means clustering

- Requires the number of clusters to be known
- Takes no account of spatial location

### Mean Shift Segmentation...

- Do not need to know the number of clusters
- Can provide spatial as well as colour segmentation
- Developed by Comaniciu & Meer 2002



Image from "Mean Shift: A Robust Approach Toward Feature Space Analysis" Dorin Comaniciu., and Peter Meer ,  
IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, VOL. 24, NO. 5, MAY 2002;

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 33

33

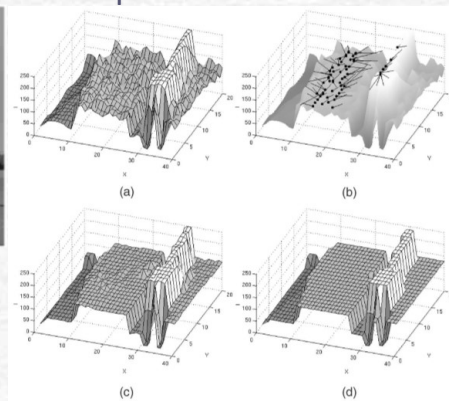
## Mean Shift for segmentation– goal

- To associate each point with a particular high-density cluster/mode in colour space.



### Move particles

- in the direction of
- the local
- increasing density



Images from "Mean Shift: A Robust Approach Toward Feature Space Analysis" Dorin Comaniciu., and Peter Meer ,  
IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, VOL. 24, NO. 5, MAY 2002;

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 34

34

## Kernel Density Estimation

- The Problem: Given a sparse dataset determine an estimate of density at each point.

$$\hat{f}_h(x) = \frac{1}{n} \sum_{i=1}^n K_h(x - x_i) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

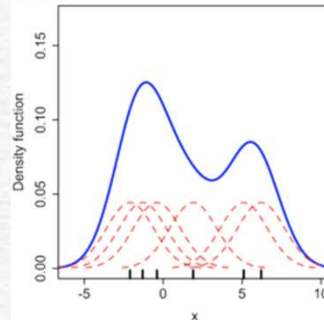
- Kernel Density  $K()$  function,  $n$  samples  $x_i$
- $h$  is the bandwidth

- Effectively

- Smooth all data samples
- Add them all together

- If the dataset is multidimensional:

$$\hat{f}(\mathbf{x}) = \frac{1}{nh^d} \sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right).$$



Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 35

35

## Kernel Density Estimation – Kernels

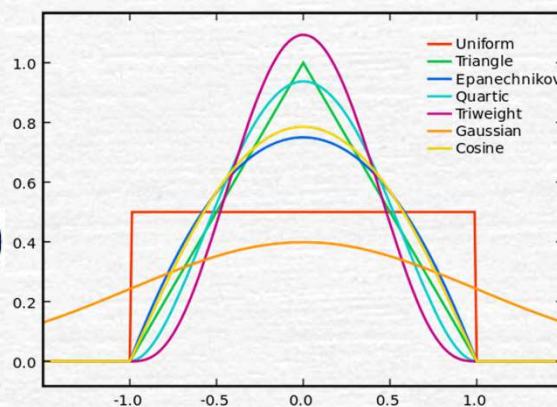
- Many different kernels.
- Kernel functions must integrate to 1.
- Typically use either

- Uniform

$$k_E(x) = \begin{cases} 1-x & 0 \leq x \leq 1 \\ 0 & x > 1, \end{cases}$$

- Gaussian

$$K_N(\mathbf{x}) = (2\pi)^{-d/2} \exp\left(-\frac{1}{2} \|\mathbf{x}\|^2\right)$$



Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 36

36

## Mean Shift – Basic Algorithm

- For each particle (pixel)
  - Estimate the local kernel density and more importantly the direction of local increasing density (the *mean shift vector*)
  - Shift the particle to the new mean.
  - Re-compute until the location stabilizes.
- Finally identify which pixels ended up in the same location
  - Mark these as members of the same cluster.
  - Determine the local mean of similar particles

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 37

37

## Mean Shift – The local kernel density

- We limit the points included in the kernel density estimate based on distance and on similar to the current point.

$$K_{h_s, h_r}(\mathbf{x}) = \frac{C}{h_s^2 h_r^2} k\left(\left\|\frac{\mathbf{x}^s}{h_s}\right\|^2\right) k\left(\left\|\frac{\mathbf{x}^r}{h_r}\right\|^2\right)$$

- We must use both a spatial kernel and a colour kernel.
- Both can be Gaussian
- Spatial kernel limits/weights the region to consider around the current point
- Colour/range kernel limits/weights the colour/intensity of the points to be included in the mean.

Maths from "Mean Shift: A Robust Approach Toward Feature Space Analysis" Dorin Comaniciu., and Peter Meer ,  
IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, VOL. 24, NO. 5, MAY 2002;

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 38

38

## Mean Shift – The mean shift vector

For radially symmetric kernels  $K(\mathbf{x}) = c_{k,d}k(\|\mathbf{x}\|^2)$

So the KDE becomes  $\hat{f}_{h,K}(\mathbf{x}) = \frac{c_{k,d}}{nh^d} \sum_{i=1}^n k\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right)$  as  $\hat{f}(\mathbf{x}) = \frac{1}{nh^d} \sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)$

We are interested in the rate of change...

$$\hat{\nabla} f_{h,K}(\mathbf{x}) \equiv \nabla \hat{f}_{h,K}(\mathbf{x}) = \frac{2c_{k,d}}{nh^{d+2}} \sum_{i=1}^n (\mathbf{x} - \mathbf{x}_i) k'\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right)$$

Set  $g(x) = -k'(x)$  so

$$\begin{aligned} &= \frac{2c_{k,d}}{nh^{d+2}} \sum_{i=1}^n (\mathbf{x}_i - \mathbf{x}) g\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right) \\ &= \frac{2c_{k,d}}{nh^{d+2}} \left[ \sum_{i=1}^n g\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right) \right] \left[ \frac{\sum_{i=1}^n \mathbf{x}_i g\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right)}{\sum_{i=1}^n g\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right)} - \mathbf{x} \right] \end{aligned}$$

Define the mean shift vector as

$$\mathbf{m}_{h,G}(\mathbf{x}) = \frac{\sum_{i=1}^n \mathbf{x}_i g\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right)}{\sum_{i=1}^n g\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right)} - \mathbf{x}$$

Maths from "Mean Shift: A Robust Approach Toward Feature Space Analysis" Dorin Comaniciu, and Peter Meer,  
IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, VOL. 24, NO. 5, MAY 2002;

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 39

39

## Mean Shift – Spatial kernel functions



Varying the Spatial radius and varying the colour radius

Regions

Based on *A Practical Introduction to Computer Vision with OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 40

40

## Mean Shift – Pros and cons

- + Do not need to know the number of clusters *a priori*.
- + Provides spatial as well as colour segmentation.
- Selection of kernel widths can be very hard.
- It is quite slow particularly if there are a lot of clusters.



Images from "Mean Shift: A Robust Approach Toward Feature Space Analysis" Dorin Comaniciu., and Peter Meer ,  
IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, VOL. 24, NO. 5, MAY 2002;  
From <http://www.comaniciu.net/Papers/MsRobustApproach.pdf>

Regions

Based on *A Practical Introduction to Computer Vision with  
OpenCV* by Kenneth Dawson-Howe © Wiley & Sons Inc. 2014

Slide 41